

Parte 1 – Entendimento de Conceitos (In-Memory e HTTP)

1. Efemeridade do In-Memory

Passos realizados:

1. A aplicação foi executada com 'dotnet run'.
2. Foram cadastrados três candidatos via Swagger (POST).
3. Os candidatos apareceram corretamente na listagem (GET).
4. A aplicação foi interrompida (Ctrl + C) e reiniciada.
5. Ao verificar novamente com GET, os dados não estavam mais presentes.

Resposta:

Os dados não permanecem após reiniciar a aplicação. Isso ocorre porque o banco In-Memory armazena os dados apenas na memória RAM, que é apagada quando a aplicação é encerrada.

Em um cenário real, isso seria um problema em sistemas que precisam persistir dados, como sistemas bancários ou de vendas.

Por outro lado, é útil em testes e desenvolvimento, pois permite rapidez e não exige banco de dados real.

2. Semântica HTTP

Teste 1 – POST com objeto vazio:

Ao enviar um POST com um objeto vazio ({}), a API geralmente retorna um erro 400 BadRequest, pois os dados obrigatórios não foram informados.

Teste 2 – DELETE em ID inexistente:

Ao tentar deletar um recurso com ID inexistente (ex: 99), a API pode retornar 404 Not Found ou 204 No Content.

Resposta ideal:

O comportamento mais adequado é retornar 404 Not Found, pois indica claramente que o recurso não foi encontrado. Em sistemas distribuídos robustos, isso é importante para garantir consistência e facilitar o tratamento de erros.

Parte 3: Reflexão sobre Sistemas Distribuídos

6. Idempotência na Prática:

Ao executar o método DELETE para o ID 1 pela primeira vez, o candidato é removido do banco de dados com sucesso.

Ao executar o DELETE novamente para o mesmo ID, a API retorna NotFound, pois o registro já não existe mais.

Em relação ao estado do banco de dados, não há mudança na segunda execução. O dado já havia sido removido anteriormente, então o estado permanece o mesmo.

Isso demonstra que o método DELETE é idempotente, pois executar a mesma operação várias vezes não altera o estado do sistema após a primeira execução.